

Lab 2: DB Console & SQL Operational Tour (70 min)

Learning Objectives

By the end of this lab you will be able to:

- Navigate every major DB Console tab against a live, loaded cluster
- Drive load with the built-in `cockroach workload` tool and watch dashboards react
- Read every `crdb_internal` table you'll need on day 1 of an on-call rotation
- Build SQL queries that reproduce DB Console answers (so you can wire them into Prometheus, alerts, or Slack bots)
- Inspect long-running jobs and active sessions

Prerequisites

- Three terminals available (you'll keep one cluster, one workload generator, and one query console running side by side)
- Lab 1 complete OR a fresh cluster — both work here

Setup

Start a 3-node demo cluster (or reuse Lab 1's):

```
scripts/crdb up          # start the 3-node cluster (skip if it is already running)
scripts/crdb sql         # open a SQL shell
```

Everything runs in Docker — see [Lab 1](#) for the cluster layout. Use `scripts/crdb.sh` on macOS, Linux, and **WSL** — WSL is Linux, so the shell script is the right one there. Only a native Windows shell (`cmd` or PowerShell) needs `scripts\crdb.bat`.

`cockroach workload` needs a connection string. There are two, and which one you use depends on where the command runs:

```
from your machine      postgresql://root@localhost:26257?sslmode=disable
from inside the cluster postgresql://root@crdb1:26257?sslmode=disable
```

`scripts/crdb run ...` executes inside node 1's container, so it uses the second form.

In **terminal B**, capture it:

```
export CRDB_URL='postgresql://root@localhost:26257/?sslmode=disable'
```

Open the Web UI URL in your browser. Leave it open in a side window.

Tasks

Part A: Tour the Web UI Without Traffic (10 min)

Click through these tabs in order, noting what's empty vs populated when the cluster is idle:

1. **Overview** — three live nodes, ~30 ranges, ~0 QPS
2. **Metrics** → **Overview dashboard** — flat lines (good — no load)
3. **Metrics** → **Hardware** — node CPU/memory/disk per node
4. **Metrics** → **SQL** — connection count = 1 (yours)
5. **Databases** — `defaultdb` exists; `system` is hidden by default
6. **SQL Activity** → **Statements** — only the queries you have run since startup
7. **SQL Activity** → **Sessions** — your active session
8. **Jobs** — system internal only (auto-stats, etc.)
9. **Hot Ranges** (Advanced Debug) — sorted by QPS, currently sleepy

Write down the **node IDs**, **localities**, and the **default replication factor** before moving on. You'll reference them in Part D.

Part B: Generate Load With `cockroach workload kv` (15 min)

`cockroach workload` is a built-in load generator with 10+ workloads — KV, MovR, TPC-C, ledger, ycsb, etc. We'll use `kv`.

1. In **terminal B**, initialize the workload schema:

```
scripts/crdb run workload init kv --drop 'postgresql://root@crdb1:26257?sslmode=disable'
```

This creates a `kv` database with one `kv` table.

2. Run a 30-second mixed workload:

```
scripts/crdb run workload run kv \  
  --duration=30s \  
  --concurrency=8 \  
  --read-percent=50 \  
  "$CRDB_URL"
```

3. While it's running, refresh the DB Console:

- **Metrics** → **Overview** — QPS jumps to ~1000/s; SQL latency shows real percentiles
- **SQL Activity** → **Statements** — `UPSERT INTO kv...` and `SELECT v FROM kv...` near the top by execution count
- **Hot Ranges** — `kv.kv` ranges now have non-zero QPS

4. Run a write-heavy workload and compare write latency:

```
scripts/crdb run workload run kv \  
  --duration=20s \  
  --concurrency=16 \  
  --read-percent=10 \  
  "$CRDB_URL"
```

In the **SQL** → **Write Latency** chart, you should see p99 climb. Compare against the read-heavy run.

5. Try a high-contention workload:

```
scripts/crdb run workload run kv \  
  --duration=20s \  
  --concurrency=16 \  
  --read-percent=0 \  
  --batch=1 \  
  --max-block-bytes=128 \  
  --cycle-length=10 \  
  "$CRDB_URL"
```

Setting `--cycle-length=10` means writes target only 10 distinct keys → very high contention. Watch **SQL Activity** → **Transactions** — contention time per transaction climbs sharply.

Part C: `crdb_internal` Operational Queries (20 min)

The DB Console is built on top of `crdb_internal`. Anything visible in the UI is queryable in SQL — and you'll want that for monitoring scripts. Run these in your demo SQL shell (terminal A).

1. Cluster-wide node health:

```
SELECT  
  count(*) FILTER (WHERE is_live)      AS live_nodes,  
  count(*) FILTER (WHERE NOT is_live)   AS dead_nodes,  
  count(*)                             AS total_nodes  
FROM crdb_internal.gossip_nodes;
```

2. Top 5 statements by total runtime — your "slow query log":

```

SELECT
  ((statistics->'statistics'->'cnt')::INT
   ((statistics->'statistics'->'svcLat'->'mean')::FLOAT * 1000)::DECIMAL(10,2) AS execs,
  left(metadata->'query', 80) AS statement
FROM crdb_internal.statement_statistics
ORDER BY (statistics->'statistics'->'cnt')::INT DESC
LIMIT 5;

```

3. Currently-running queries longer than 1 second:

```

SELECT node_id, session_id, age(now(), start) AS run_time, left(query, 80)
FROM crdb_internal.cluster_queries
WHERE start < now() - INTERVAL '1 second';

```

Most of the time this is empty. Trigger one with a deliberately slow query in terminal B and re-run.

4. Hot ranges:

```

-- Where the data lives and who holds each lease
SELECT table_name, range_id, lease_holder, range_size_mb
FROM [SHOW CLUSTER RANGES WITH TABLES, DETAILS]
WHERE database_name = 'kv'
ORDER BY range_size_mb DESC
LIMIT 5;

```

Per-range QPS is not available in SQL. `crdb_internal` exposes `ranges` and `ranges_no_leases`, and neither carries a queries-per-second column (there is no `cluster_replicas` table either). Hot-range traffic comes from **DB Console** → **Advanced Debug** → **Hot Ranges**, or the `/_status/hotranges` HTTP endpoint. What you *can* get from SQL is the distribution — how many ranges a table has and which node holds each lease.

5. Active sessions and what they're doing:

```

SELECT node_id, application_name, client_address,
       coalesce(active_queries, '<idle>') AS active_query
FROM crdb_internal.cluster_sessions
WHERE status = 'active'
ORDER BY node_id;

```

6. Storage usage per node:

```

SELECT node_id,
       (metrics->'capacity.available')::DECIMAL / 1e9 AS available_gb,
       (metrics->'capacity.used')::DECIMAL / 1e9 AS used_gb
FROM crdb_internal.kv_store_status
ORDER BY node_id;

```

7. Transaction contention events (run after the high-contention workload from Part B step 5):

```
SELECT count(*) AS contention_events,
       coalesce(max(contention_duration), '0s') AS max_contention
FROM crdb_internal.transaction_contention_events;
```

Part D: Build a "Custom Dashboard" From SQL (10 min)

Imagine your team wants a daily Slack message: nodes up, hottest table, contention level, top slow statements. Compose those answers as SQL — exactly what you'd put in a cron.

1. **Compose this one big query** that gives a complete cluster snapshot:

```
WITH
  liveness AS (
    SELECT count(*) FILTER (WHERE is_live) AS up,
           count(*) FILTER (WHERE NOT is_live) AS down
    FROM crdb_internal.gossip_nodes
  ),
  top_table_by_size AS (
    SELECT table_name, range_size_mb
    FROM [SHOW CLUSTER RANGES WITH TABLES, DETAILS]
    WHERE table_name IS NOT NULL
    ORDER BY range_size_mb DESC NULLS LAST
    LIMIT 1
  ),
  contention AS (
    SELECT count(*) AS contention_events
    FROM crdb_internal.transaction_contention_events
    WHERE collection_ts > now() - INTERVAL '5 minutes'
  )
SELECT
  (SELECT up FROM liveness) AS nodes_up,
  (SELECT down FROM liveness) AS nodes_down,
  (SELECT table_name FROM top_table_by_size) AS biggest_range_table,
  (SELECT contention_events FROM contention) AS recent_contention;
```

2. **Schedule it conceptually** — what's your alerting threshold for each column?

Part E: Jobs — Long-Running Background Work (10 min)

Schema changes, backups, restores, changefeeds, decommissions, and IMPORTs all run as **jobs**. Here's how to find and manage them.

1. **List recent jobs:**

```
SELECT job_id, job_type, status, created,
       coalesce(finished, now()) - created AS duration,
       left(description, 60) AS description
FROM [SHOW JOBS]
ORDER BY created DESC
LIMIT 10;
```

2. **Trigger a long-ish job** — building an index on a moderate-sized table:

```
CREATE INDEX kv_v_idx ON kv.kv(v);
```

3. Immediately query the jobs table:

```
SELECT job_id, status, fraction_completed, running_status
FROM [SHOW JOBS]
WHERE job_type IN ('SCHEMA CHANGE', 'NEW SCHEMA CHANGE')
ORDER BY created DESC LIMIT 1;
```

For our tiny `kv` table this is over in a second, but on a real production table you'd see `fraction_completed` climb from 0 toward 1.

4. Pause and resume a job (try this with a backup later if you want — for now just learn the syntax):

```
-- PAUSE JOB <job_id>;
-- RESUME JOB <job_id>;
-- CANCEL JOB <job_id>;
```

Part F: Session Inspection and Cancellation (5 min)

You suddenly need to kill someone's runaway query. Here's how.

1. Find expensive sessions:

```
SELECT node_id, session_id, application_name, age(now(), session_start) AS up,
       left(active_queries, 100) AS query
FROM crdb_internal.cluster_sessions
WHERE status = 'active' AND active_queries != '';
```

2. Cancel a query:

```
-- CANCEL QUERY '<query_id>';
-- CANCEL SESSION '<session_id>;'
```

(Don't actually cancel your own — it disconnects you.)

Cleanup

Stay in the cluster for Lab 3 if you'd like — the `kv` data won't interfere.

Otherwise:

```
DROP DATABASE kv CASCADE;
\q
```

The cluster keeps running between labs — that is the point of it being persistent. To wipe everything and start fresh at any time:

Lab 2 Deliverables

□ **Web UI toured:** visited every major tab and noted what's empty vs populated □ **Workload generated:** ran read-heavy, write-heavy, and high-contention KV workloads □ **crdb_internal queried:** pulled node liveness, top statements, hot ranges, sessions, storage, contention □ **Custom dashboard query:** composed a single SQL that yields a Slack-ready cluster snapshot □ **Jobs inspected:** triggered a schema change, watched it as a job

Challenge Exercises

1. **Build an "is the cluster healthy?" SQL** that returns a single boolean. Define healthy as: all expected nodes live, under-replicated ranges = 0, no statement p99 latency above 100 ms in the last minute.
2. **Reproduce the Hot Ranges page in SQL.** The DB Console's Hot Ranges page sorts ranges by QPS. Write a single SELECT that gives the same answer.
3. **Compare the kv table after the read-heavy vs write-heavy workloads.** Look at `crdb_internal.kv_store_status.metrics`. Which storage metrics changed the most?

Reference

Need	View
Node liveness	<code>crdb_internal.gossip_nodes</code> , <code>crdb_internal.gossip_liveness</code>
Range placement / leases	<code>crdb_internal.ranges_no_leases</code> , <code>[SHOW RANGES FROM TABLE t]</code>
Top statements	<code>crdb_internal.statement_statistics</code>
Running queries	<code>crdb_internal.cluster_queries</code>
Open sessions	<code>crdb_internal.cluster_sessions</code>
Long-running jobs	<code>[SHOW JOBS]</code> (or <code>crdb_internal.jobs</code>)
Contention	<code>crdb_internal.transaction_contention_events</code>
Storage / disk	<code>crdb_internal.kv_store_status</code>
Built-in workloads	<code>cockroach workload --help</code>